114 計算機組織 Homework 4_3

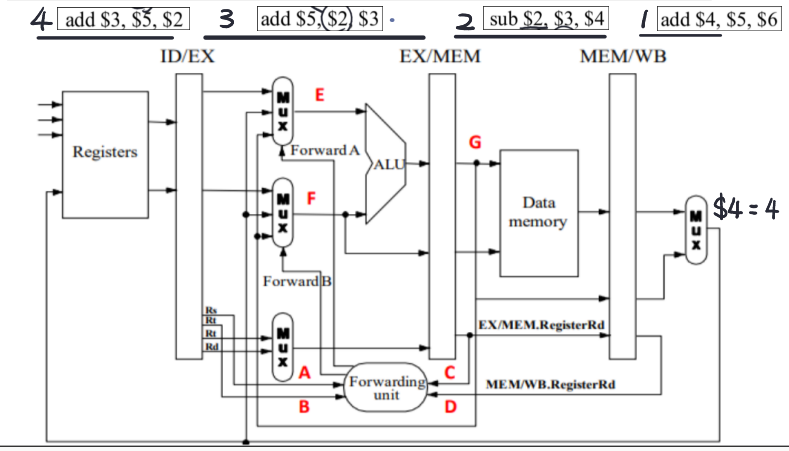
1.(15%) Assume that the code will be processed by the pipelined datapath shown below.

1 add $4, $5, $6 $4 = 3+1 = 4
2 sub $2, $3, $4 $2 = 5-4 = 1
3 add $5, $2, $3 $5 = 1+5 = 6
4 add $3, $5, $2

Assume that the initial value of registers shown in Table 1. In the **5th clock**, please calculate the A, B, C, D, E, F and G.

Table 1

$2	$3	$4	$5	$6
4	5	2	3	1



	1	2	3	4	5	6	7	8
add	IF	ID	EX	MEM	WB			
sub		IF	ID	EX	MEM	WB		
add			IF	ID	EX	MEM	WB	
add				IF	ID	EX	MEM	WB

ANS: (A) 2 Number of $2 of I3 =
(B) 3 Number of $3 of I3 =
(C) 2 Number of $2 of I2 =
(D) 4 Number of $4 of I1 =
(E) 1 Number of $2 from I3
(F) 5 Number of $3 from I3
(G) 1 Result of I2 (5-2)

2. (6%) Please answer the following questions with True/False:

(a) (2%) Pipelining does not reduce the latency of a single task, but it improves the throughput of an entire workload.

ANS: True

(b) (2%) An instruction takes less time to execute on a pipelined processor than on a non-pipelined processor(all other aspects of the processors being the same).

ANS: False, 因為 Pipeline 處理器的時鐘週期取決於耗時最多的步驟，所以會使用更長的時間。

(c) (2%) A load-use data hazard occurs because the pipeline flushes the instructions behind.

ANS: False, 是因為 load 的資料要寫到 WB 才能取得，相隔 2 個週期的 use 指令無法取得此來自未來的值。

3. (25%) The importance of having a good branch predictor depends on how often conditional branches are executed. Together with branch predictor accuracy, this will determine how much time is spent stalling due to mispredicted branches. In this exercise, assume that the breakdown of dynamic instructions into various instruction categories is as follows:

R-Type	beq	jump	lw	sw
40%	20%	5%	20%	15%

Also, assume the following branch predictor accuracies:

Always-Taken	Always-Not-Taken	2-bit
70%	30%	95%

(1) (5%) Stall cycles due to mispredicted branches increase the CPI. What is the extra CPI due to mispredicted branches with the always-taken predictor? Assume that branch outcomes are determined in the **MEM** stage, that there are no data hazards, and that no delay slots are used.

Decide in MEM = 3 clock punishment

Extra CPI = CPI × 20% × 30% × 3 = 0.18 CPI

A: 0.18

(2) (5%) Repeat (1) for the "always-not-taken" predictor.

Extra CPI = CPI × 20% × 70 % × 3 = 0.42 CPI

A: 0.42

(3) (5%) Repeat (1) for the "2-bit predictor" predictor.

Extra CPI = CPI × 20% × 5 % × 3 = 0.03 CPI

A: 0.03

(4) (5%) With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that **replaces a branch instruction with an ALU instruction**? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

$CPI_{old} = 1 + 20\% \times 5\% \times 3 = 1.03$

$CPI_{new} = 1 + 20\% \times 5\% \times 3 \times \frac{1}{2} = 1.015$

SPEED UP = $\frac{1.03}{1.015} = 1.014778$

A: 1.014778

(5) (5%) With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that **replaced each branch instruction with two ALU instructions**? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.

$CPI_{old} = 1 + 20\% \times 5\% \times 3 = 1.03$

$CPI_{new} = 1 + (1 \times 20\% \times \frac{1}{2} \times 1) + (1 \times 20\% \times \frac{1}{2} \times 5\% \times 3) = 1.115$

SPEED UP = $\frac{1.03}{1.115} = 0.9237$

A: 0.9237

5.(20%) To understand the cost/complexity/performance trade-offs of forwarding in a pipelined processor. Refer to pipelined datapaths from Figure 4.45, assume that, all the instructions executed in a processor, the following fraction of these instructions have a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the instruction that consumes the result (1st instruction that follows the one that produces the result, 2nd instruction that follows, or both). Assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so "EX to 3rd" and "MEM to 3rd" dependencies are not counted because they cannot result in data hazards. Also, assume that the CPI of the processor is 1 if there are no data hazards.

(1) "EX to 1st only 10%" indicates that the data produced in the EX stage is required only by the next instruction, and 10% represents that this scenario occurs in 10% of all executed instructions

(2) "EX to 2nd only 20%" indicates that the data produced in the EX stage is required only by the second instruction, and 20% represents that this scenario occurs in 20% of all executed instructions

EX to 1^{st} Only	MEM to 1^{st} Only	EX to 2^{nd} Only	MEM to 2^{nd} Only	EX to 1^{st} and MEM to 2^{nd}	Other RAW Dependences
5%	10%	15%	10%	10%	10%

5.1. (6%) If we use no forwarding, what fraction of cycles are we stalling due to data hazards?

$(5\% \times 2 + 10\% \times 2 + 15\% \times 1 + 10\% \times 1 + 10\% \times 2 + 10\% \times 0) = 0.75\%$ (cycles)

$0.75 \div (1 + 0.75) = 0.429$

A: 42.9%

5.2. (7%) If we use full forwarding (forward all results that can be forwarded), what fraction of cycles are we stalling due to data hazards?

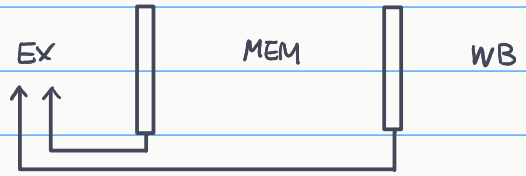
Load-use : MEM to 1^{st} = 10% $0.1 / (1 + 0.1) = 0.09$

A: 9%

5.3. (7%) Let us assume that we cannot afford to have three-input Muxes that are needed for full forwarding. We must decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). Which of the two options results in fewer data stall cycles?

EX / MEM = $\underbrace{10\% \times 2}_{\text{MEM-1st}} + \underbrace{15\%}_{\text{EX-2nd}} + \underbrace{10\%}_{\text{MEM-2nd}} + 10\% = 55\%$

MEM / WB = $\underbrace{5\%}_{\text{EX-1st}} + \underbrace{10\%}_{\text{MEM-1st}} + \underbrace{10\%}_{\text{Both}} = 25\%$

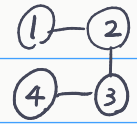


A: From MEM/WB

6. (16%) This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: NT, T, T, NT, NT, T, T, NT

6.1. (8%)What is the accuracy of the two-bit predictor for the first 4 branches in this pattern, assuming that the predictor starts off in the bottom left state from Figure 4.63 (predict not taken, dark gray)?

Attempt	1	2	3	4	5	6	7	8
Guess	NT	NT	NT	T	NT	NT	NT	T
Correct	✓	X	X	X	✓	X	X	X
State	4	3	2	3	4	3	2	3



1 ÷ 4 = 25%

ANS: 25%

6.2.(8%) What is the accuracy of the two-bit predictor if this pattern is repeated forever, assuming that the predictor starts off in the top left state from Figure 4.63 (predict taken, dark blue)?

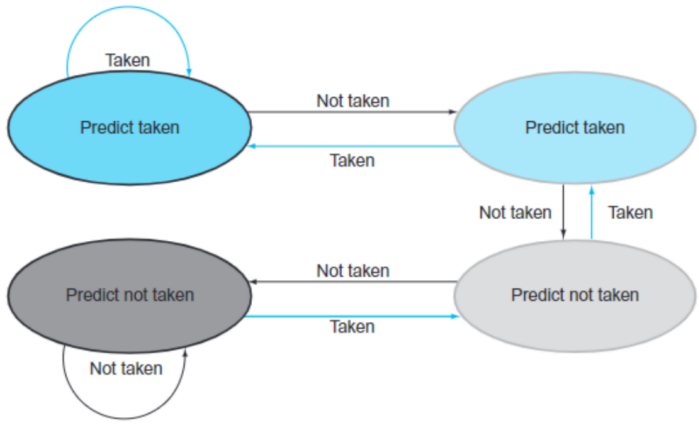
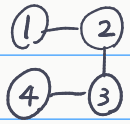


FIGURE 4.63 The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted only once. The 2 bits are used to encode the four states in the system. The 2-bit scheme is a general instance of a counter-based predictor, which is incremented when the prediction is accurate and decremented otherwise, and uses the mid-point of its range as the division between taken and not taken.

Attempt	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Guess	T	T	T	T	T	NT	T	T	T	NT	T	T	T	NT	T	T
Answer	NT	T	T	NT	NT	T	T	NT	NT	T	T	NT	NT	T	T	NT
Correct	X	✓	✓	X	X	X	✓	X	X	X	✓	X	X	X	✓	X
State	1	2	1	2	3	2	1	2	3	2	1	2	3	2	1	2

$\frac{2}{8}$ = 25 %



ANS: 25 %

7. (5%) Consider a 5-stage pipelined datapath with forwarding. For the first five cycles of execution, specify which signals are asserted in **each cycle** by **hazard detection** and **forwarding units** in the following Figure 4.55 and Figure 4.60 (PCWrite, IF/IDWrite, Forward_A, and Forward_B)

lw r4,4(r5)
sub r6,r2,r4
sw r3,0(r4)
or r3,r4,r3
lw r5,0(r4)

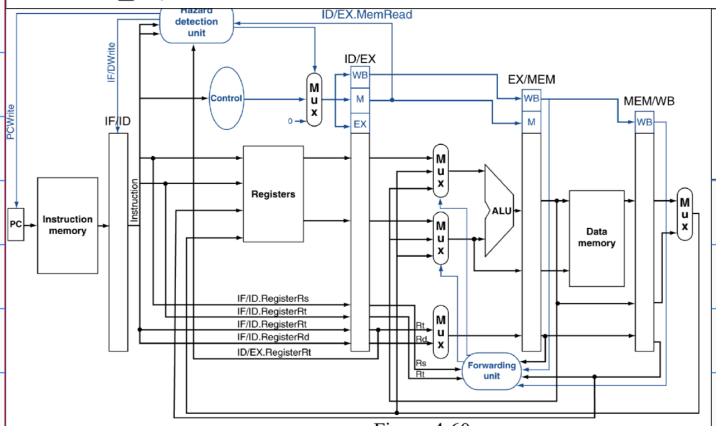


Figure 4.60

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

FIGURE 4.55 The control values for the forwarding multiplexors in Figure 4.54. The signed immediate that is another input to the ALU is described in the *Elaboration* at the end of this section.

Lines	Executed Instructions	Pipeline Cycles				
		1	2	3	4	5
1	lw r4, 4(r5)	IF	ID	EX	MEM	WB
2	sub r6, r2, r4		IF	ID	<nop>	EX
3	sw r3, 0(r4)			IF	<nop>	ID
4	or r3, r4, r3				<nop>	IF
5	lw r5, 0(r4)					
Signal	PCwrite	1	1	0	1	1
	IF/ID write	1	1	0	1	1
	ForwardA	X	X	00	X	00
	ForwardB	X	X	X	X	01